FATEC • PAM | EXPO SDK 57 | REACT NATIVE 0.86 | TYPESCRIPT 6.0

ENGENHARIA DE SENSORES

MANUAL DE ENGENHARIA E ARQUITETURA DE SOFTWARE MOBILE

PAM FITNESS

Sistema de monitoramento biomecânico e telemetria esportiva com smartphone no braço, acelerômetro tridimensional, rastreamento GPS geodésico, diário visual por câmera e persistência local.

⚡ Acelerômetro FSM

Detecção de repetições e cadência muscular em tempo real via magnitude tridimensional euclidiana e debounce temporal de 700ms.

🌐 GPS & Pacing

Cálculo de distância incremental por trigonometria esférica de Haversine e conversão para ritmo atlético internacional (min/km).

📷 Auditoria Visual

Captura de fotos de séries, anilhas e maquinário via CameraView nativa com persistência offline através do AsyncStorage.

Instituição: Faculdade de Tecnologia (FATEC)
Disciplina: Programação de Aplicativos Mobile (PAM)
Projeto: PAM Fitness — Monitor de Treino por Hardware

Versão da Documentação: 1.0.0 (Consolidada)
Data de Conclusão: Outubro de 2026
Status: Aprovado para Avaliação e Produção

ÍNDICE GERAL DE CONTEÚDO

Manual Técnico Consolidado

1. Visão Geral, Arquitetura e Escopo do Sistema	**Página 3**
1.1 Proposta de Valor e Conceito do Smartphone no Braço	1.1
1.2 Casos de Uso Primários do Aplicativo	1.2
1.3 Identidade Visual e Design System (Dark Mode Atlântico)	1.3
2. Estruturas de Árvore e Mapeamento Arquitetural	**Páginas 4-5**
2.1 Árvore Completa de Diretórios e Arquivos do Projeto	2.1
2.2 Árvore de Navegação e Telas (Expo Router)	2.2
2.3 Árvore de Fluxo de Dados e Camadas de Estado	2.3
3. Engenharia e Física dos Sensores de Hardware	**Páginas 6-8**
3.1 Sensor de Movimento (Acelerômetro) & Magnitude Euclidiana	3.1
3.2 Máquina de Estados Finita (FSM), Debounce e Cadência	3.2
3.3 Sensor de GPS (Haversine e Pacing) & Sensor de Câmera	3.3
4. Catálogo Detalhado de Hooks e Funções React / Expo	**Páginas 9-10**
4.1 Hooks Nativos do React (useState, useEffect, useRef, useCallback)	4.1
4.2 Hooks Específicos do Expo (useRouter, useFocusEffect, useCameraPermissions)	4.2
4.3 Matriz de Otimização e Prevenção de Vazamento de Memória	4.3
5. Catálogo de Serviços, Persistência Local e Tipagem TypeScript	**Páginas 11-12**
5.1 Funções Utilitárias Geográficas e Formatações (locationService.ts)	5.1
5.2 Persistência Offline com AsyncStorage (storage.ts)	5.2
5.3 Contratos de Dados e Tipagem Estrita (workout.ts)	5.3
6. Handlers de Interface, Telas e Fluxo do Treino	**Páginas 13-14**
6.1 Handlers das Telas (Treino Ativo, Câmera e Histórico)	6.1
6.2 Diagrama Integrado do Ciclo de Vida da Sessão	6.2
7. Boas Práticas, Mobile-First e Avaliação FATEC	**Página 15**
7.1 Gestão Energética, Continuous Native Generation (CNG) e Testes	7.1

ℹ️ Resumo Executivo da Engenharia do Projeto

O **PAM Fitness** é uma aplicação móvel desenvolvida no ecossistema **React Native** com **Expo SDK 57** e **TypeScript**. O projeto transforma o smartphone em um dispositivo vestível esportivo autossuficiente ao ser fixado no braço via braçadeira elástica, dispensando smartwatches adicionais. Integra simultaneamente três sensores nativos de hardware: o **acelerômetro linear** (para contagem biomecânica de repetições e cadência muscular), o **receptor GPS** (para telemetria de distância e ritmo de pacing) e o **sensor CMOS de câmera** (para registro fotográfico e auditoria de cargas e anilhas). Todos os dados são consolidados localmente via `AsyncStorage` garantindo funcionamento *offline-first* ininterrupto.

🎯 Eliminação de Acessórios

Substitui sensores caros de pulso usando o processamento interno do próprio smartphone na braçadeira esportiva.

🛡️ Arquitetura Robusta

Desacoplamento estrito entre UI (Expo Router), controle (Hooks), lógica pura (Services) e dados (TypeScript).

01 Visão Geral, Arquitetura e Escopo do Sistema

1.1. Proposta de Valor e Conceito do Smartphone no Braço

No treinamento resistido (musculação) e funcional, a quantificação precisa de esforço, repetições e cadência depende tipicamente de smartwatches caros ou anotações manuais que interrompem a dinâmica da sessão. O **PAM Fitness** converte o smartphone preso a uma braçadeira no braço em um dispositivo esportivo completo:

- **Contador Biomecânico de Repetições:** Analisa os vetores de aceleração linear do membro em três dimensões, discriminando as fases concêntrica (subida/contração) e excêntrica (descida/alongamento).
- **Computador de Bordo de Corrida:** Monitora o deslocamento geográfico contínuo por satélite, calculando velocidade instantânea, velocidade média e ritmo de *pacing* (min/km).
- **Diário Visual de Séries:** Registra instantaneamente fotos de maquinário e cargas com notas da série.

1.2. Casos de Uso Primários do Aplicativo

CASO DE USO	FLUXO DO USUÁRIO	MÓDULOS E SENSORES ACIONADOS
1. Treino de Força	Atleta prende o celular no braço, inicia o treino e executa repetições (rosca, elevação, supino).	`useWorkoutTimer`, `useArmMotionTracker` (leitura do acelerômetro a cada 100ms via `expo-sensors`).
2. Cardio / Corrida	Na corrida ou aquecimento, alterna para a aba de Pacing e monitora velocidade e ritmo em min/km.	`useLocationTracker` (GPS de alta precisão via `expo-location`), `locationService` (Fórmula de Haversine).
3. Auditoria Visual	Após a série pesada, clica no atalho da câmera, fotografa as anilhas e anota a carga do exercício.	`expo-camera` (componente `CameraView`), `useCameraPermissions`, estrutura `WorkoutPhoto`.
4. Fechamento e Histórico	Finaliza o treino, revisa o resumo no modal e confirma o salvamento permanente offline.	`storage.ts` (serialização JSON em `AsyncStorage`), tela `historico.tsx` para auditoria.

1.3. Identidade Visual e Tendências de Design (Dark Mode Atlântico)

O design do aplicativo adota o **Minimalismo Esportivo de Alto Contraste**, otimizado para economia de energia em telas OLED e leitura sob iluminação intensa de academias:

- **Paleta Escura Atlântica:** Fundo `#09090B` e cartões `#141416` com bordas `#27272A`.
- **Neon Volt (`#CCFF00`):** Acento visual de alta energia para métricas biomecânicas (reps, cadência RPM).
- **Cyber Cyan (`#38BDF8`):** Acento geodésico para telemetria de navegação (pacing, distância em km, GPS).
- **Tipografia Tabular de Alta Escala:** Dígitos centrais entre 48px e 72px com `tabular-nums` legíveis a 40 cm.
- **Micro-indicadores de Status:** Pílulas táteis luminosas indicando sinal de satélite e prontidão de sensores.

02 Estruturas de Árvore e Mapeamento Arquitetural

Para facilitar a compreensão global da solução, o sistema é mapeado em três estruturas hierárquicas: a **árvore de arquivos** (abaixo), a **árvore de navegação** e a **árvore de estado** (página seguinte).

2.1. Árvore Completa de Diretórios e Arquivos do Projeto

```
App PAM/ # Raiz do projeto React Native (Expo SDK 57)
├── docs/ # Documentação técnica e especificações acadêmicas
│   ├── README.md # Visão geral, arquitetura e escopo
│   ├── SENSORES.md # Engenharia física e matemática dos 3 sensores
│   ├── FUNCOES_REACT.md # Catálogo explicativo de hooks React e Expo
│   └── GUIA_DE_CHAMADAS.md # Catálogo de assinaturas e fluxo de execução
├── src/ # Código-fonte TypeScript da aplicação
│   ├── app/ # Telas nativas gerenciadas pelo Expo Router
│   │   ├── _layout.tsx Layout # Roteador em abas inferiores (Tabs nativas)
│   │   ├── index.tsx Tela # Painel de treino ativo, repetições e pacing
│   │   ├── camera.tsx Tela # Visor da câmera, flash e anotação de série
│   │   └── historico.tsx Tela # Lista de sessões passadas e galeria de fotos
│   ├── constants/
│   │   └── theme.ts # Paleta Dark Mode, espaçamentos e raios de borda
│   ├── hooks/ # Lógica de controle e abstração de hardware
│   │   ├── useWorkoutTimer.ts Hook # Cronômetro atômico com start/pause/resume
│   │   ├── useArmMotionTracker.ts Hook # Acelerômetro, magnitude e FSM de repetições
│   │   └── useLocationTracker.ts Hook # GPS contínuo, Haversine e cálculo de pacing
│   ├── services/ # Camada de serviços puros e persistência
│   │   ├── locationService.ts Serviço # Matemática esférica, conversão km/h e pacing
│   │   └── storage.ts Serviço # Operações CRUD offline via AsyncStorage
│   └── types/ # Contratos de tipagem estrita
│   ├── workout.ts Tipos # Interfaces LocationPoint, WorkoutSession, etc.
│   └── declarations.d.ts # Tipagem de assets e estilos globais
├── app.json # Manifesto Expo, permissões nativas e plugins CNG
└── package.json # Especificação de dependências e scripts de execução
```

Papel Arquitetural de Cada Diretório

DIRETÓRIO / ARQUIVO	RESPONSABILIDADE PRINCIPAL	PADRÃO ARQUITETURAL APLICADO
`src/app/`	Telas e rotas nativas	File-based routing (Expo Router). Isolamento completo entre telas e roteador central.
`src/hooks/`	Lógica reativa e sensores	Custom Hooks. Encapsula listeners de hardware com funções determinísticas de cleanup.
`src/services/`	Cálculos puros e storage	Pure Services e DAO (Data Access Object) offline sobre AsyncStorage.
`src/types/`	Contratos TypeScript	Single Source of Truth para tipagem de telemetria, coordenadas e sessões salvas.
`src/constants/`	Tokens visuais	Design Tokens consolidados (Colors, Spacing, Radius, Typography).

2.2. Árvore de Navegação e Hierarquia de Telas (Expo Router)

O roteamento utiliza abas nativas e modais contextuais para manter o treino acessível sem perda de estado:

```
[RootLayout: src/app/_layout.tsx] # Configura TabBar inferior escura
│
├──➤ Aba 1: "index" (Treino Ativo)
│  ├── Card do Cronômetro Principal (useWorkoutTimer)
│  ├── Seletor de Modo: [ Modo Braço / Reps ] vs [ Modo GPS & Pacing ]
│  ├── Card de Métricas de Repetições (Magnitude G, Cadência RPM, FSM)
│  ├── Card de Métricas de Corrida (Pace instantâneo, Distância km, Vel. média)
│  ├── Atalho Rápido para Captura de Foto (router.push('/camera'))
│  └── [Modal de Conclusão de Treino] # Resumo antes de gravar no storage
│
├──➤ Aba 2: "camera" (Câmera de Séries)
│  ├── Visor Nativo CameraView (Facing Frontal/Traseiro, Torch Flash)
│  ├── Controle de Permissão useCameraPermissions()
│  └── [Modo Preview Pós-Disparo]
│  ├── Campo de Texto para Anotação da Carga/Exercício
│  ├── Botão Repetir Foto (Descarta URI temporária)
│  └── Botão Salvar Foto (Persiste via storage.ts e navega)
│
└──➤ Aba 3: "historico" (Histórico & Auditoria)
├── Recarregamento Automático em Foco (useFocusEffect)
├── Cards de Sessões Salvas (Data, Duração, Reps, Distância, Pacing)
├── Miniaturas de Fotos Anexadas à Sessão
├── Ações: Excluir Sessão Individual ou Limpar Histórico Total
└── [Modal de Foto em Tela Cheia] # Inspeção detalhada de anilhas/cargas
```

2.3. Árvore de Fluxo de Dados e Camadas de Estado

O fluxo de dados transita em uma via unidirecional desde a recepção física dos sensores até a persistência offline:

```
CAMADA 1: HARDWARE NATIVO DO SMARTPHONE
├── Acelerômetro MEMS ► Gera vetores x, y, z a cada 100ms (10 Hz)
├── Receptor GNSS/GPS ► Gera coordenadas lat, lon, velocidade e precisão a cada 1s (1 Hz)
└── Sensor Óptico CMOS ► Captura bitmap JPEG comprimido (qualidade 0.85)
│
CAMADA 2: HOOKS CUSTOMIZADOS DE ESTADO E PROCESSAMENTO
├── useArmMotionTracker ► Magnitude Euclidiana + FSM Pico/Vale + Debounce 700ms
├── useLocationTracker ► Filtragem de saltos + Haversine incremental + Pacing
└── useWorkoutTimer ► Contagem atômica de segundos via setInterval protegido
│
CAMADA 3: SERVIÇOS PUROS E MATEMÁTICA
├── locationService.ts ► calculateDistanceKm, metersPerSecondToKmh, formatInstantPace
└── storage.ts ► getWorkoutSessions, saveWorkoutSession, deleteWorkoutSession
│
CAMADA 4: PERSISTÊNCIA LOCAL (OFFLINE FIRST)
└── @react-native-async-storage/async-storage ► Chave fixa: '@app_pam:workouts_v1'
│
CAMADA 5: APRESENTAÇÃO E AUDITORIA (UI)
├── src/app/index.tsx ► Renderização reativa de repetições e ritmo em 60fps
└── src/app/historico.tsx ► Listagem histórica indexada com recuperação de fotos
```

03 Engenharia e Física dos Sensores de Hardware

O núcleo técnico do **PAM Fitness** consiste no processamento matemático em tempo real de três sensores físicos integrados ao smartphone: acelerômetro, GPS e câmera.

3.1. Sensor de Movimento (Acelerômetro) — Celular no Braço

3.1.1. Princípio Físico e Eixos Espaciais

O acelerômetro é um sensor eletromecânico MEMS (*Micro-Electro-Mechanical Systems*) que mede a aceleração linear decomposta em três eixos ortogonais:

- **Eixo X:** Aceleração lateral (esquerda/direita em relação à tela).
- **Eixo Y:** Aceleração longitudinal (cima/baixo no chassi do aparelho).
- **Eixo Z:** Aceleração perpendicular ao plano da tela (frente/trás).

Em repouso na superfície da Terra, o sensor não acusa aceleração nula, mas sim a força da aceleração estática da gravidade ($1.0g \approx 9.81\ m/s^2$) apontando em direção ao centro do planeta.

3.1.2. Magnitude Resultante Tridimensional

A orientação angular do celular na braçadeira esportiva varia dependendo de como o atleta o veste (no bíceps, antebraço, voltado para frente ou para o lado). Monitorar um eixo isolado geraria contagens erráticas. O sistema soluciona essa variabilidade calculando a **Magnitude Euclidiana Tridimensional**:

$$\mathbf{Magnitude} = \sqrt{(x^2 + y^2 + z^2)}$$

Onde x, y e z representam as acelerações instantâneas medidas em unidades de g pelo sensor. Sendo a magnitude uma grandeza escalar tridimensional, qualquer flexão muscular concêntrica ou extensão excêntrica produzirá uma variação mensurável, independentemente do alinhamento angular do aparelho.

> 📐 **Por que a Magnitude Tridimensional é Superior a Eixos Isolados?**
>
> Se um algoritmo monitorasse apenas o eixo Y, ao realizar uma rosca direta com o celular colocado na lateral do braço, a aceleração se projetaria majoritariamente nos eixos X e Z, fazendo o app falhar na contagem. A magnitude euclidiana consolida os três vetores em um escalar invariante a rotações espaciais.

3.1.3. Controles Manuais de Ajuste Fino

Para assegurar a máxima fidedignidade ao atleta, além da contagem automática da máquina de estados, o hook `useArmMotionTracker` expõe duas funções de ajuste com estabilidade referencial via `useCallback`:

- `addManualRep()`: Permite ao usuário somar 1 repetição manualmente caso tenha executado um movimento de amplitude muito curta que não tenha atingido o limiar.
- `removeManualRep()`: Permite decrementar uma repetição caso um movimento brusco acidental tenha sido computado, com proteção para nunca assumir valores negativos (`Math.max(0, prev - 1)`).

3.2. Máquina de Estados Finita (FSM), Debounce e Cadência

3.2.1. Algoritmo da Máquina de Estados de Repetições

O fluxo de detecção do ciclo biomecânico completo é governado por uma Máquina de Estados Finita (FSM) com filtragem de ruído implementada no hook `useArmMotionTracker`:

1 **ESTADO: REPOUSO / AGUARDANDO PICO**

O membro do atleta encontra-se na posição inicial do exercício. A magnitude resultante oscila próxima a **1.0g** (gravidade estática da Terra). Estado interno: `repStateRef = 'esperando_pico'`.

▼ Aceleração concêntrica: Magnitude ≥ 1.25g

2 **ESTADO: SUBIDA DETECTADA (AGUARDANDO VALE)**

O músculo se contrai para erguer o peso. O braço acelera para cima, elevando a força resultante além do limiar superior de **1.25g**. O estado transiciona para `repStateRef = 'esperando_vale'` e `motionState = 'subindo'`.

▼ Desaceleração excêntrica: Magnitude ≤ 0.85g E intervalo decorrido > 700ms

3 **ESTADO: REPETIÇÃO CONTADA (+1 REP)**

Ao alcançar a extensão controlada no ponto inferior, a força cai abaixo de **0.85g**. Se decorridos mais de **700ms** da repetição anterior, o contador é incrementado, a cadência é recalculada e o estado retorna a aguardar novo pico.

Por que o filtro de Debounce de 700ms é indispensável?

Na biomecânica do exercício resistido, nenhuma repetição com amplitude articular completa (*Range of Motion*) é executada fisiologicamente em menos de 0,7 segundos. Se um pico e um vale ocorressem em 200ms ou 300ms, isso corresponderia a tremor muscular por fadiga (clônus) ou choque mecânico de anilhas. O debounce temporal de 700ms descarta 100% dessas falsas detecções.

3.2.2. Cálculo de Cadência em Tempo Real (RPM)

A cadência reflete a velocidade de execução em repetições por minuto. O aplicativo mantém um vetor deslizante de timestamps das repetições ocorridas nos últimos 30 segundos:

$$\mathbf{RPM} = (\text{ Repetições nos últimos 30 segundos }) \times 2$$

Ao multiplicar por 2, projeta-se o ritmo exato para 60 segundos (1 minuto). Repetições ocorridas há mais de 30.000 ms são descartadas automaticamente através do filtro: `recentRepsTimestampsRef.current.filter(t => t > cutoff)`.

Esse modelo dinâmico permite ao praticante manter a consistência de velocidade mesmo quando a fadiga muscular se acumula ao final de séries extensas.

3.3. Sensor de Localização (GPS) e Ritmo de Pacing

3.3.1. Captura de Satélite e Cálculo Geodésico (Fórmula de Haversine)

O hook `useLocationTracker` utiliza a biblioteca nativa `expo-location` configurada com alta resolução (`Accuracy.High`, amostragem de 1 segundo ou 2 metros). A distância entre coordenadas consecutivas é computada pela **Fórmula de Haversine**:

$$a = \sin^2(\Delta lat / 2) + \cos(lat_1) \cdot \cos(lat_2) \cdot \sin^2(\Delta lon / 2)$$

$$c = 2 \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{(1 - a)}) \quad | \quad \textbf{Distância (km)} = R \cdot c \quad (\text{com } R = 6.371 \text{ km})$$

Onde Δlat e Δlon são as diferenças de coordenadas convertidas em radianos (graus × π / 180). Saltos anômalos de sinal de satélite superiores a 50 metros em 1 segundo (equivalente a 180 km/h) são rejeitados pelo filtro protetor.

3.3.2. Ritmo de Pacing Esportivo (Minutos por Quilômetro)

No atletismo e na corrida de rua, a velocidade não é avaliada em km/h, mas sim pelo **pacing** (tempo necessário para percorrer 1 km):

- Se a velocidade instantânea do GPS for inferior a 0.5 m/s (~1.8 km/h), o app identifica repouso ou semáforo fechado e exibe o indicador neutro: `--'--"`.
- Para velocidades válidas, computa-se: `Segundos por km = 1000 / velocidade (m/s)`, decomposto em minutos e segundos restantes formatados no padrão internacional (ex: `05'24" /km`).
- O ritmo médio da sessão é calculado dividindo os segundos totais decorridos pela distância acumulada em quilômetros.

3.4. Sensor de Câmera (Expo Camera)

A auditoria visual de equipamentos e cargas é realizada pelo componente nativo `CameraView` no Expo SDK 57:

- **Controle de Permissões:** Orquestrado via `useCameraPermissions()` com diálogo nativo amigável.
- **Alternância de Hardware:** Alterna entre a lente traseira (`back`) para fotografar anilhas e frontal (`front`) para autorretratos, com controle contínuo do LED da lanterna (`enableTorch`).
- **Disparo e Auditoria:** O disparo via `takePictureAsync({ quality: 0.85 })` gera o arquivo otimizado, permite anotação de texto (ex: "4x12 com 30kg cada lado") e associa a foto permanentemente à sessão ativa.

04 Catálogo Detalhado de Hooks e Funções React / Expo

Este capítulo apresenta o catálogo exaustivo de todos os hooks de gerenciamento de estado e ciclo de vida utilizados no PAM Fitness, demonstrando a conformidade técnica com o modelo funcional do React 19 e Expo SDK 57.

4.1. Catálogo Completo do Hook `useState`

HOOK / ARQUIVO	VARIÁVEIS DECLARADAS	TIPO TYPESCRIPT	RESPONSABILIDADE NO PAM FITNESS
`useWorkoutTimer.ts`	`seconds, isActive, isPaused`	`number, boolean, boolean`	Contagem atômica de tempo de treino e estados de execução/pausa do cronômetro.
`useArmMotionTracker.ts`	`reps, axes, intensityG, cadenceRpm, motionState`	`number, {x,y,z}, number, number, 'repouso'|'subindo'|'descendo'`	Métricas biomecânicas reativas: contagem de reps, aceleração nos 3 eixos, força G e cadência RPM.
`useLocationTracker.ts`	`permissionGranted, metrics`	`boolean|null, LocationMetrics`	Status de autorização de GPS e métricas agregadas (pacing, distância em km, velocidades e rota).
`src/app/index.tsx`	`activeTab, finishModalVisible, sessionPhotos`	`'braco'|'pacing', boolean, WorkoutPhoto[]`	Alternância entre painel de braço e GPS, exibição do modal de finalização e fotos capturadas na sessão.
`src/app/camera.tsx`	`facing, flashMode, capturedPhotoUri, photoNote, isCapturing`	`CameraType, 'off'|'on', string|null, string, boolean`	Controles da câmera, URI da foto capturada em preview, anotação da série e bloqueio de cliques duplos.
`src/app/historico.tsx`	`sessions, loading, selectedPhotoUri`	`WorkoutSession[], boolean, string|null`	Lista de treinos históricos, estado de carregamento e URI da foto aberta em tela cheia no modal.

4.2. Gestão de Ciclo de Vida e Limpeza de Efeitos com `useEffect`

O hook `useEffect` é reservado estritamente para conexões de hardware e timers, com **funções de limpeza (cleanup) mandatórias** para prevenir vazamentos de memória e drenagem de bateria:

- **Temporizador (`useWorkoutTimer.ts`):** Instancia um `setInterval` a cada 1.000ms condicionado a `isActive && !isPaused`. A função de limpeza executa `clearInterval(intervalRef.current)` ao pausar ou desmontar.
- **Acelerômetro (`useArmMotionTracker.ts`):** Quando `isActive === true`, configura o intervalo em 100ms via `Accelerometer.setUpdateInterval(100)` e conecta o ouvinte. No cleanup, chama `subscriptionRef.current.remove()`, desligando o sensor físico imediatamente.
- **Rastreamento GPS (`useLocationTracker.ts`):** Inicia a escuta em `Location.watchPositionAsync` com precisão alta. No cleanup, cancela a assinatura nativa de satélite.

4.3. Preservação de Alta Frequência com `useRef`

Modificar a propriedade `ref.current` **não causa re-renderização** da tela. Esse padrão é essencial em aplicações com sensores de hardware para evitar sobrecarga de renderização a cada 100ms:

REFERÊNCIA	ARQUIVO DE ORIGEM	POR QUE UTILIZA USEREF E NÃO USESTATE?
`lastRepTimeRef`	`useArmMotionTracker.ts`	Guarda o timestamp milissegundo da última rep para o debounce de 700ms. Evita renderizar a árvore React a cada milissegundo computado.
`repStateRef`	`useArmMotionTracker.ts`	Armazena o estado interno da FSM (`esperando_pico` vs `esperando_vale`) sem causar repintura da interface a cada amostra intermediária.
`recentRepsTimestampsRef`	`useArmMotionTracker.ts`	Vetor de timestamps das repetições recentes para o cálculo de cadência RPM.
`lastPointRef` & `accumulatedDistanceRef`	`useLocationTracker.ts`	Armazena a coordenada anterior e acumula a distância matemática de Haversine sem atrasos de ciclo de renderização.
`cameraRef`	`src/app/camera.tsx`	Guarda a referência direta ao nó nativo `CameraView` para invocar imperativamente o método `takePictureAsync()` .

4.4. Estabilidade Referencial com `useCallback`

O `useCallback` memoriza referências de funções passadas para botões e componentes filhos, prevenindo re-renderizações espúrias:

- `start()` , `pause()` , `resume()` e `reset()` em `useWorkoutTimer` .
- `resetReps()` , `addManualRep()` e `removeManualRep()` em `useArmMotionTracker` .
- `requestPermission()` e `resetMetrics()` em `useLocationTracker` .
- `loadSessions()` em `src/app/historico.tsx` para invocação segura no hook de foco.

4.5. Hooks Específicos do Expo e Expo Router

- **`useRouter` (de `expo-router`):** Provê despacho imperativo de rotas na pilha de navegação nativa. Exemplo: `router.push('/camera')` para transição até o visor fotográfico e `router.back()` para retornar ao treino.
- **`useFocusEffect` (de `expo-router`):** Dispara uma ação sempre que a tela em questão se torna visível ao usuário. Utilizado na tela `historico.tsx` para recarregar a listagem de treinos do AsyncStorage assim que o atleta toca na aba de Histórico.
- **`useCameraPermissions` (de `expo-camera`):** Hook oficial que gerencia o diálogo de autorização nativa de câmera no Android e iOS.

05 Catálogo de Serviços, Persistência Local e Tipagem TypeScript

Toda a computação matemática e o acesso a dados residem no diretório `src/services/`, mantendo a camada de visualização desacoplada de regras algorítmicas.

5.1. Módulo Geográfico e de Pacing (`src/services/locationService.ts`)

FUNÇÃO	ASSINATURA E TIPOS	COMPORTAMENTO E RETORNO
`calculateDistanceKm`	`(lat1: number, lon1: number, lat2: number, lon2: number): number`	Aplica a trigonometria esférica de Haversine com o raio da Terra de 6.371 km. Retorna a distância percorrida em quilômetros.
`metersPerSecondToKmh`	`(speedMps: number | null): number`	Multiplica a velocidade bruta em m/s pelo fator 3.6. Retorna a velocidade em km/h com 1 casa decimal (retorna 0 se nulo ou negativo).
`formatInstantPace`	`(speedMps: number | null): string`	Converte velocidade em segundos/km (1000 / v) e formata como `MM'SS"`. Se a velocidade for menor que 0.5 m/s, retorna `--'--"`.
`formatAveragePace`	`(totalSeconds: number, distanceKm: number): string`	Divide o tempo total decorrido em segundos pela distância percorrida em km. Retorna o ritmo médio ponderado no formato `MM'SS"`.
`formatDuration`	`(totalSeconds: number): string`	Formata o total de segundos inteiros para o padrão legível de cronômetro: `MM:SS` (ou `HH:MM:SS` para treinos acima de 1 hora).

5.2. Módulo de Persistência Offline (`src/services/storage.ts`)

O armazenamento de treinos é *Offline-First*, gravando os dados no disco interno do smartphone sem necessidade de conexão de rede sob a chave `@app_pam:workouts_v1`:

FUNÇÃO	ASSINATURA TYPESCRIPT	OPERAÇÃO NO ASYNCSTORAGE
`getWorkoutSessions`	`(): Promise<WorkoutSession[]>`	Lê a chave local, desserializa o JSON e ordena o array decrescentemente pela data de início (treinos recentes primeiro). Retorna [] em falhas.
`saveWorkoutSession`	`(session: WorkoutSession): Promise<boolean>`	Recupera a lista atual, insere a nova sessão no índice zero e serializa de volta em JSON. Retorna `true` em caso de sucesso.
`deleteWorkoutSession`	`(id: string): Promise<boolean>`	Filtra o array excluindo a sessão com o ID especificado e grava o vetor atualizado em disco. Retorna booleano de confirmação.
`clearAllSessions`	`(): Promise<boolean>`	Remove a chave completa do AsyncStorage via `removeItem`, zerando todo o histórico local com segurança.

5.3. Contratos de Dados e Tipagem Estrita (`src/types/workout.ts`)

A integridade estrita entre os dados medidos pelos sensores e as entidades persistidas é assegurada por interfaces TypeScript consolidadas:

```typescript
// 1. Telemetria Geodésica (GPS)
export interface LocationPoint {
  latitude: number;
  longitude: number;
  altitude: number | null;
  speed: number | null;
  timestamp: number;
}

export interface LocationMetrics {
  currentPace: string;
  averagePace: string;
  distanceKm: number;
  currentSpeedKmh: number;
  averageSpeedKmh: number;
  accuracyMeters: number | null;
  route: LocationPoint[];
}

// 2. Amostras de Hardware
export interface MotionSample {
  x: number;
  y: number;
  z: number;
  magnitude: number;
  timestamp: number;
}
```

```typescript
// 3. Métricas Biomecânicas
export interface ArmMotionMetrics {
  reps: number;
  cadenceRpm: number;
  intensityG: number;
  motionState: 'repouso' | 'subindo' | 'descendo';
  lastRepTimestamp: number | null;
}

// 4. Auditoria Fotográfica
export interface WorkoutPhoto {
  id: string;
  uri: string;
  timestamp: number;
  repsAtCapture: number;
  note?: string;
}

// 5. Sessão Consolidada em Disco
export interface WorkoutSession {
  id: string;
  title: string;
  startTime: number;
  endTime: number;
  durationSeconds: number;
  totalReps: number;
  totalDistanceKm: number;
  averagePace: string;
  averageSpeedKmh: number;
  photos: WorkoutPhoto[];
  notes?: string;
}
```

> 🛡️ **Vantagens da Tipagem Estrita no Ecossistema Mobile**
>
> O uso de interfaces TypeScript completas assegura que nenhuma propriedade essencial (como `durationSeconds` ou `totalReps`) seja omitida no momento em que a sessão é serializada no AsyncStorage, evitando erros silenciosos de *undefined* durante a renderização no Histórico.

06 Handlers de Interface, Telas e Fluxo do Treino

As interações do atleta com a interface acionam funções manipuladoras (*handlers*) que coordenam os hooks de sensores e a camada de navegação.

6.1. Tela de Treino Ativo (`src/app/index.tsx`)

- **`handleStartWorkout()`:** Dispara `timer.start()`, reinicializa reps com `arm.resetReps()`, zera métricas com `location.resetMetrics()` e ativa os ouvintes dos sensores em background.
- **`handleNavigateToCamera()`:** Invoca `router.push('/camera')`, permitindo ao atleta registrar fotografias de cargas sem interromper a contagem do cronômetro.
- **`handleOpenFinishModal()`:** Pausa o cronômetro temporariamente com `timer.pause()` e exibe o modal de resumo com os totais de tempo, repetições, cadência e distância.
- **`handleConfirmSaveWorkout()`:** Consolida os dados em uma instância de `WorkoutSession`, persiste no banco local com `saveWorkoutSession(newSession)`, reseta as métricas e exibe alerta de confirmação.
- **`handleCancelFinish()`:** Fecha o modal de conclusão e retoma o cronômetro com `timer.resume()` caso o usuário decida continuar a série.
- **`handleDiscardWorkout()`:** Fecha o modal e limpa todas as variáveis da sessão sem gravar nada no AsyncStorage.

6.2. Tela de Câmera de Séries (`src/app/camera.tsx`)

- **`handleToggleFacing()`:** Alterna a propriedade `facing` da câmera entre a lente traseira (`back`) e a frontal (`front`).
- **`handleToggleFlash()`:** Alterna a lanterna de LED do aparelho (`enableTorch`) entre ligado e desligado.
- **`handleTakePicture()`:** Invoca `cameraRef.current.takePictureAsync({ quality: 0.85 })`, bloqueia cliques múltiplos com `isCapturing = true`, salva o arquivo temporário e abre o preview de revisão.
- **`handleRetakePhoto()`:** Descarta a foto recém-capturada e retorna ao visor ao vivo para nova tentativa.
- **`handleSavePhoto()`:** Constrói o objeto `WorkoutPhoto` contendo o ID gerado, a URI da foto, a nota explicativa digitada pelo usuário e a quantidade de reps no instante do disparo, gravando os dados e redirecionando para `/historico`.

6.3. Tela de Histórico e Auditoria (`src/app/historico.tsx`)

- **`loadSessions()`:** Função assíncrona que lê as sessões ordenadas via `getWorkoutSessions()` e atualiza o estado local para renderização dos cards esportivos.
- **`handleDeleteSession(id: string)`:** Exibe uma caixa de diálogo nativa de confirmação (`Alert.alert`) e, após aprovação do atleta, remove a sessão especificada via `deleteWorkoutSession(id)`.
- **`handleClearAll()`:** Solicita confirmação explícita para remoção de todo o banco de histórico com `clearAllSessions()`.

6.4. Diagrama Integrado do Ciclo de Vida da Sessão

O diagrama abaixo detalha a jornada de ponta a ponta da telemetria, desde o momento em que o smartphone é colocado na braçadeira até o arquivamento definitivo da sessão no histórico:

```
[ ATLETA PRENDE CELULAR NA BRAÇADEIRA ]
│
├──► Toca em "INICIAR TREINO"
│ │
│ ├──► [ useWorkoutTimer ] Inicia contagem atômica de segundos (1 Hz)
│ ├──► [ useArmMotionTracker ] Liga Acelerômetro (10 Hz) e monitora magnitude G
│ └──► [ useLocationTracker ] Liga GPS contínuo e calcula Haversine/Pacing
│
├──► Durante o Treino (Concorrente)
│ ├── Executa exercício: FSM detecta contração/extensão muscular (+1 Rep, Cadência RPM)
│ ├── Corre ou caminha: GPS triangula coordenadas e converte para MM'SS"/km
│ └── Audita carga: Toca em Câmera, fotografa anilhas com CameraView e adiciona notas
│
└──► Toca em "FINALIZAR TREINO"
│
├──► Modal de resumo exibe consolidação completa de estatísticas
├──► Confirmação: Serializa objeto WorkoutSession em JSON
├──► AsyncStorage: Grava na chave permanente '@app_pam:workouts_v1'
└──► Aba Histórico: Recarrega via useFocusEffect e exibe card permanente
```

> 💡 **Resiliência de Estado Durante o Treino**
>
> Como o cronômetro e os listeners de aceleração e geolocalização utilizam `useRef` e subscrições desacopladas da tela, o atleta pode transitar livremente entre a visualização de braço e a visualização de pacing — e inclusive abrir o visor da câmera para fotografar uma anilha — sem que o cronômetro zere ou que repetições já contadas sejam perdidas.

07 Boas Práticas, Mobile-First e Avaliação FATEC

Este capítulo sintetiza as diretrizes de engenharia de software mobile e os critérios acadêmicos aplicados no desenvolvimento do projeto **PAM Fitness** na disciplina de **Programação de Aplicativos Mobile (PAM)** da FATEC.

7.1. Gestão Energética e Eficiência de Bateria

- **Desligamento Imediato de Sensores (Cleanup):** Nenhum sensor permanece ativo em segundo plano quando o cronômetro não está em execução. Pausar ou finalizar cancela a assinatura de GPS e o listener do acelerômetro.
- **Amostragem Otimizada (100ms e 2m):** O acelerômetro opera a 10 Hz (100ms), taxa ideal para a velocidade muscular humana que economiza até 70% de processamento em comparação a taxas de 60 Hz.
- **Dark Mode com Fundo Preto Verdadeiro (`#09090B`):** Em telas OLED/AMOLED, os pixels pretos permanecem totalmente apagados, minimizando o consumo elétrico durante treinos prolongados.

7.2. Continuous Native Generation (CNG) e Expo SDK 57

- As pastas `android/` e `ios/` **não sofrem edições manuais**; são geradas de forma reproduzível a partir do manifesto `app.json` e de config plugins oficiais.
- Todas as permissões nativas de hardware — como `ACCESS_FINE_LOCATION` e `CAMERA` — são declaradas formalmente no `app.json` com mensagens justificativas transparentes ao usuário.
- Arquitetura moderna com **React Native 0.86** e **React 19**, aproveitando o compilador otimizado e TypeScript estrito.

7.3. Checklist para Demonstração e Verificação Técnica

ITEM DE DEMONSTRAÇÃO	PROCEDIMENTO DE TESTE NO SMARTPHONE	CRITÉRIO DE SUCESSO ESPERADO
1. Acelerômetro e Reps	Prender o celular no braço e executar 5 repetições de rosca bíceps com ritmo controlado.	Contador atinge exatamente 5 repetições com transições "subindo", "descendo" e "repouso".
2. Filtragem de Ruído (Debounce)	Sacudir o braço rapidamente ou efetuar pequenos tremores musculares em menos de 700ms.	O contador rejeita tremores e não incrementa repetições falsas.
3. GPS e Ritmo de Pacing	Caminhar ou correr em área aberta com sinal de satélite ativo.	Velocidade calculada em km/h e pacing formatado em `MM'SS" /km`.
4. Captura de Câmera e Nota	Acessar a aba da câmera, fotografar um halter, digitar a anotação da série e salvar.	Foto armazenada com nota e exibida na galeria da sessão no Histórico.
5. Persistência Offline	Fechar o app no multitarefa do sistema operacional e reabri-lo.	Todas as sessões e fotos salvas permanecem íntegras no AsyncStorage.

> 🎓 **Conclusão e Avaliação Acadêmica**
>
> O **PAM Fitness** demonstra a aplicação prática dos conceitos fundamentais da disciplina: integração com APIs de sensores nativos de hardware, gerenciamento determinístico de ciclo de vida com React Hooks, arquitetura limpa com tipagem estrita em TypeScript, design system atlético mobile-first e persistência offline.